

Herald



Herald — commons_storage Module Guide (Operator / Developer)

Field	Value
Revision	1
Created	2026-05-31
Last modified	2026-05-31
Status	active
Status summary	Nano-detail per-module reference for <code>commons_storage</code> — Herald's L1 storage layer (spec V4 §10): the Postgres connection wrapper (pgx pool via <code>digital.vasic.database</code>), the <code>WithTenantContext</code> RLS / <code>FORCE ROW LEVEL SECURITY</code> tenant-isolation helper (carrying the E14 falsifiability lesson), the 15 embedded SQL migrations (000001..000015) bundled via <code>//go:embed migrations/*.sql</code>, the pherald <code>migrate up/status/down/validate</code> CLI, and how the Postgres-backed background task queue + Redis client are wired one layer up in <code>commons_infra</code>. ANTI-BLUFF: every section documents only what the source under <code>commons_storage/</code> (plus the <code>commons_infra</code> boot seam) actually does as of this revision.
Issues	(none specific to this guide) bump when <code>pherald migrate down/validate</code> graduate from honest-501 stubs to live implementations, when the upstream <code>pkg/migration</code> placeholder fix (HRD-082) lets <code>RunMigrations</code> collapse back to a one-line <code>runner.Apply()</code>, and when the <code>commons_storage/go.mod</code>-internal River/Redis wiring referenced in the <code>storage.go</code> package doc actually lands inside this module (today it lives in <code>commons_infra</code>).
Continuation	

Table of contents

- [§1. Overview](#)
- [§2. The Postgres connection wrapper](#)
- [§3. Tenant isolation — `WithTenantContext`, RLS, and the E14 lesson](#)
- [§4. Embedded migrations — the `//go:embed` bundle](#)
- [§5. The migration runner](#)
- [§6. The pherald migrate CLI](#)
- [§7. The background task queue + Redis \(wired in `commons\_infra`\)](#)

- [§8. Environment variables](#)
 - [§9. How flavors consume commons_storage](#)
 - [§10. Operator migrations runbook](#)
 - [§11. Troubleshooting](#)
 - [§12. Testing notes](#)
 - [§13. References](#)
-

§1. Overview

`commons_storage` (Go package `storage`, module path `github.com/vasic-digital/herald/commons_storage`) is Herald's **L1 storage layer** per spec V4 §10. Its package doc (top of `storage.go`) states its scope verbatim:

Package `storage` is Herald's L1 storage layer (`commons_storage`) per spec V3 §10. It owns the Postgres connection wrapper, RLS context helpers, embedded migrations, and Redis tenant-namespacing client.

In the shipped source it owns three things directly:

1. **The Postgres connection wrapper** (`postgres.go`) — `Open / OpenWithPool / ConfigForHerald / ParseDSN`, thin adapters over `digital.vasic.database/pkg/postgres` (a `pgx v5` pool) per the §11.4.74 catalogue-check "extend, don't reimplement" pivot.
2. **The RLS tenant-context helper** (`postgres.go WithTenantContext + storage.go SetTenantContext`) — the load-bearing tenant-isolation seam (§3).
3. **The embedded SQL migrations** (`storage.go //go:embed migrations/*.sql, migrator.go runner`) — 15 files, 000001..000015 (§4–§5).

The **River-style background task queue and the Redis client** are referenced in the `storage.go` package doc as belonging to this layer, but as of this revision the live wiring lives one layer up in `commons_infra` (`boot.go / clients.go`); see §7. The unimplemented `MigrationDriver` interface + `NewMigrationDriver` in `storage.go` are scaffold stubs (they return HRD-010 not-implemented errors) — the **live** migration path is `RunMigrations` (§5), not that interface.

Key third-party / submodule dependencies (`commons_storage/go.mod`):
`digital.vasic.database` (replaced to `../submodules/database` — the `pgx` pool + migration runner), `github.com/jackc/pgx/v5 v5.9.2`, `github.com/redis/go-redis/v9 v9.7.3`, `github.com/google/uuid`.

§2. The Postgres connection wrapper

All four constructors live in `postgres.go` and wrap `digital.vasic.database/pkg/postgres` rather than opening a raw `pgx` pool here.

Function	Signature	Behaviour
Open	<code>Open(ctx, cfg *postgres.Config) (db.Database, error)</code>	Connects and returns the universal <code>db.Database</code> Helix-stack abstraction. Defaults <code>ApplicationName</code> to "herald" when empty. Caller owns defer

Function	Signature	Behaviour
OpenWithPool	OpenWithPool(ctx, cfg) (db.Database, *pgxpool.Pool, error)	client.Close(). Errors on nil cfg. Open's twin that also returns the raw *pgxpool.Pool for callers (e.g. the pherald Runner's pg adapters) that drive raw pgx.Query/pgx.Exec. The pool is owned by the wrapper — callers MUST db.Close(), never pool.Close().
ConfigForHerald	ConfigForHerald(host, port, user, password, dbName) *postgres.Config	Builds a *postgres.Config with Herald-friendly defaults: Driver="postgres", SSLMode="disable" (local-dev), ApplicationName="herald".
ParseDSN	ParseDSN(dsn string) (*postgres.Config, error)	Converts a postgres://user:pass@host:port/dbname[?sslmode=...] URL into the typed config. Default port 5432. Only the sslmode query parameter is honoured; others are silently ignored by design. Rejects empty DSN, non-postgres(ql) schemes, invalid ports.

The Foundation default pool config (documented in Open's doc comment) is: max-conns = cpu × 4 with a floor of 4 / ceiling of 64, statement cache enabled, application_name="herald". Overrides flow straight through the cfg argument into digital.vasic.database.

ParseDSN is the bridge that lets operators supply a single HERALD_PG_DSN env var instead of five typed knobs — it is exactly what pherald migrate consumes (§6).

§3. Tenant isolation — WithTenantContext, RLS, and the E14 lesson

This is the load-bearing security seam of the module.

§3.1 The model

Herald is multi-tenant. Every multi-tenant table carries a tenant_id UUID column and a PostgreSQL **Row-Level Security (RLS)** policy of the shape:

```
ALTER TABLE <t> ENABLE ROW LEVEL SECURITY;
ALTER TABLE <t> FORCE ROW LEVEL SECURITY;           -- see §3.3
CREATE POLICY <t>_isolation ON <t>
    USING      (tenant_id =
                current_setting('app.tenant_id')::uuid)
    WITH CHECK (tenant_id =
                current_setting('app.tenant_id')::uuid);
```

The policy keys on the `app.tenant_id` GUC (a session/transaction-scoped runtime parameter). `storage.go`'s `SetTenantContext(tenantID)` returns the SQL string `SET LOCAL app.tenant_id = '<uuid>'`; per spec §16 runtime code **MUST** run this before any `SELECT/INSERT` against multi-tenant tables, otherwise the policies fail-closed (zero rows / blocked writes).

§3.2 WithTenantContext — the runtime entry point

`WithTenantContext(ctx, database, tenantID, fn)` (in `postgres.go`) is the helper every runtime caller uses. Inside one transaction it:

1. Begins a transaction on database.
2. Runs `SET LOCAL ROLE herald_app` — drops the bootstrap user's superuser privileges down to the `herald_app` role (which is `NOBYPASSRLS` per migration `000001` and holds the `CRUD` grants from `000008`).
3. Runs `SET LOCAL app.tenant_id = '<uuid>'` (inline string — `SET LOCAL` does **not** accept \$1 parameters; `UUID` format is constrained by `uuid.UUID.String()` so injection is impossible).
4. Invokes `fn(tx)`.
5. Commits if `fn` returns `nil`, rolls back otherwise. A commit failure is returned.

Both `SET LOCAL`s are **transaction-scoped**, so isolation is automatically reset at commit/rollback and nested transactions each set their own context.

§3.3 The E14 falsifiability lesson (FORCE ROW LEVEL SECURITY + SET LOCAL ROLE)

There are **two** independent traps that each make tenant isolation a silent bluff, and both are guarded:

- **Trap A — owner bypass.** A table's **owner role bypasses RLS by default** even with policies defined. The Foundation integration tests connect as `herald` (the DB owner per the quickstart compose's `POSTGRES_USER`), so without `FORCE ROW LEVEL SECURITY` the `tenant_isolation` policy is silently bypassed. This was caught **2026-05-20** by `TestPostgresStore_RLSTenantIsolation` (`commons_constitution/postgres_integration_test.go`): tenant B could read tenant A's `constitution_state` row. Migration `000008_force_rls.up.sql` is the fix — it adds `FORCE ROW LEVEL SECURITY` to **every** RLS-guarded table created so far and grants `herald_app` the necessary `CRUD`.
- **Trap B — superuser bypass (the E14 lesson).** Even with `FORCE` set, a **SUPERUSER** connection bypasses RLS entirely. The bootstrap `POSTGRES_USER` (quickstart's `herald`) is typically a superuser. So `WithTenantContext`'s `SET LOCAL ROLE herald_app` step is load-bearing: without it, calls from the bootstrap user bypass RLS regardless of `FORCE`, and tests asserting tenant isolation would **PASS while production was wide open**. This was discovered **2026-05-20** by the §107 **E14** round-trip test in `commons_storage/storage_integration_test.go`. The `WithTenantContext` doc comment records it verbatim as the anti-bluff rationale.

Operator takeaway. Tenant isolation only holds when **BOTH** are true: the table has `FORCE ROW LEVEL SECURITY` (every Herald multi-tenant migration sets it) **AND** the connection runs as the non-superuser `herald_app` role (which `WithTenantContext` enforces via `SET LOCAL ROLE`). Bypassing

WithTenantContext and querying multi-tenant tables directly as the bootstrap superuser silently disables isolation.

§3.4 Tables that are deliberately NOT RLS-guarded

background_tasks, background_task_events, task_resource_snapshots, and dead_letter_tasks (migrations 000009/000013/000014) carry **no** tenant_id + RLS by design — they are dispatcher-internal queue/monitoring infrastructure. Tenant context lives **inside** the task's JSON payload and is enforced one level up by Herald service code that constructs+enqueues the task, not by the queue table. The schema_migrations tracking table is also un-tenanted (it is global schema metadata).

§4. Embedded migrations — the //go:embed bundle

storage.go embeds the entire migrations/ directory at compile time:

```
//go:embed migrations/*.sql
var migrationsFS embed.FS

func MigrationsFS() embed.FS { return migrationsFS }
```

MigrationsFS() exposes the embedded FS so callers (the runner, a doctor command, a schema dump) can read the canonical SQL without filesystem access — the SQL ships **inside the binary**, so a deployed pherald needs no migration files on disk.

There are **15** migrations, each a paired NNNNNN_<name>.up.sql + .down.sql. The migration set as of this revision:

Version	File stem	What the .up.sql does
000001	init_core	Creates the herald_migrator (BYPASSRLS) and herald_app (NOBYPASSRLS) roles, the uuidv7() SQL function (time-ordered PKs — Postgres ≤16 ships none), and the canonical tenants table.
000002	idempotency_keys	idempotency_keys (PK (tenant_id, idempotency_key), 24h expires_at, FILLFACTOR=80) + RLS/FORCE + idem_isolation policy. Spec §4.3.
000003	subscribers	subscribers (logical party: handle, display_name, kind ∈ human/agent/service, roles[], metadata), subscriber_aliases (per-channel channel+channel_user_id), and agent_tokens — all RLS/FORCE. Spec §7.1 + §7.5.
000004	channel_addresses	channel_addresses (per-tenant outbound destinations: address_url, tags[], priority_floor, health columns) + GIN tag index + RLS/FORCE. Spec §6.
000005	inbound_pipeline	The big atomic inbound bundle: webhook_sources, inbound_messages, thread_refs,

Version File stem**What the .up.sql does**

	quarantined_messages, dead_letters,workable_items, outbound_dedup, email_suppressions, report_publications — each RLS/ FORCE. Spec §32 + §15.2 + §5.4 + §12 + §8.3 + §35.
000006 constitution_state	constitution_state (per- (tenant,rule,subject) verdict-of- record, UPSERTed by the Runner) + append- only constitution_audit (with row- level no-UPDATE / no-DELETE policies). Spec §42.1.2.
000007 constitution_bindings	constitution_bindings (per-tenant per- rule mode ladder: 0=allow / 1=warn / 2=enforce) + RLS. Redis-cached at M3 (60s TTL). Spec §42.1.4.
000008 force_rls	Adds FORCE ROW LEVEL SECURITY to every RLS table created so far + grants herald_app CRUD + USAGE on public. The Trap-A fix (§3.3).
000009 background_tasks	background_tasks + background_task_events backing the upstream digital.vasic.background TaskRepository (columns map 1:1 to the BackgroundTask struct db: tags). No tenant_id/RLS by design (§3.4). HRD-010 Task 5.
000010 outbound_delivery_evidence	outbound_delivery_evidence — per- tenant SEND-time audit trail; channel_message_id MUST be the provider-returned ID (Telegram message_id, Slack ts), not a Herald UUID (§107 guard). RLS/FORCE. Spec §11.0 + §16.
000011 claude_code_sessions	claude_code_sessions — operator- shared Claude Code session anchor (UUID + path + last dispatch). Uses the fixed HeraldSystemTenant UUID so RLS still applies uniformly. Spec §33.2.
000012 events_processed	events_processed — inbound idempotency 30-day archive (Redis SETNX is the hot path; this is the audit/replay tier). RLS keyed on app.current_tenant_id. Spec §32.2.
000013 task_resource_snapshots	task_resource_snapshots — append- only per-task CPU/mem/IO/net/fd time series for the ResourceMonitor. FK→background_tasks ON DELETE CASCADE. No tenant_id/RLS (§3.4). HRD-089.
000014 dead_letter_tasks	

Version File stem

000015 subscriber_alias_username

What the .up.sql does

dead_letter_tasks — snapshot table for the upstream MoveToDeadLetter contract (full JSONB task snapshot for reprocessing). FK→background_tasks ON DELETE CASCADE. No tenant_id/RLS (§3.4). HRD-090.

ALTER TABLE subscriber_aliases ADD COLUMN username TEXT + partial (channel, username) index — the per-channel @handle for notification @-tagging (distinct from channel_user_id). Inherits RLS via the FK to subscribers. docs/design/PARTICIPANT_ATTRIBUTION.md §4d.

Note on the test comment drift. migrations_test.go's comment says "exactly these 14 migrations" but its expectedNames slice already lists all 15 (through 000015_subscriber_alias_username). The slice — and the 15 files on disk — are authoritative; the prose comment is stale and worth fixing when that file is next touched.

Down migrations exist for every up migration so the planned migrate down path is safe (§6 / §10).

§5. The migration runner

migrator.go implements a small, Postgres-native runner. There is **no** dependency on golang-migrate in the shipped code — the spec (§9.6) names it as the intended tool, but the live runner is hand-rolled (see the HRD-082 note below).

Function

LoadEmbeddedMigrations()
([]migration.Migration,
error)

RunMigrations(ctx,
database) ([]int, error)

applyMigration(ctx,
database, m)

CurrentVersion(ctx,
database) (int, error)

Behaviour

Walks migrationsFS, parses each NNNNNN_<name>.{up,down}.sql, pairs ups with downs by version, and returns them **sorted ascending by version**. Skips a .down.sql with no matching .up.sql. Errors on a filename lacking the .up/ .down suffix or the NNN_ prefix.

Loads the bundle, inits the schema_migrations tracking table (migration.NewRunner(database, "schema_migrations").Init), reads already-applied versions, and applies each **pending** migration in its own transaction via applyMigration. Returns the versions applied **this run** (already-applied ones are skipped — idempotent).

Runs m.Up then INSERT INTO schema_migrations (version, name, applied_at) VALUES (\$1,\$2,\$3) in one transaction; commits on success, rolls back otherwise.

SELECT COALESCE(MAX(version),0) FROM schema_migrations. Returns 0 on a fresh DB where

Function

Behaviour

the tracking table does not yet exist (translates SQLSTATE 42P01 undefined_table → 0 rather than surfacing a confusing error). Drives pherald migrate status.

Why a hand-rolled runner (HRD-082). RunMigrations is a Postgres-native re-implementation because the upstream digital.vasic.database/pkg/migration v0 uses ? placeholders in its INSERT (a SQLite/MySQL convention) which pgx rejects. The doc comment tracks this as **HRD-082**: once the submodule emits pg-native \$N placeholders, this re-implementation collapses back into a one-line runner.Apply().

The tracking table is always named schema_migrations.

§6. The pherald migrate CLI

pherald migrate(pherald/cmd/pherald/migrate.go) drives commons_storage.RunMigrations/CurrentVersion from the command line. Four subcommands are registered; **two are live, two are honest-501 stubs**:

Subcommand	Status	Behaviour
pherald migrate up	LIVE	Reads HERALD_PG_DSN, ParseDSN → Open → RunMigrations; prints applied N migration(s).
pherald migrate status	LIVE	Reads HERALD_PG_DSN, ParseDSN → Open → CurrentVersion; prints schema version: N.
pherald migrate down	501 stub	Returns a helpful "not yet implemented — destructive op per §9.1 requires operator authorisation; open an HRD in docs/Issues.md" error. NOT a generic Cobra "unknown command".
pherald migrate validate	501 stub	Returns "not yet implemented — open an HRD in docs/Issues.md to request schema-drift detection".

No silent defaults. Both live subcommands **require** the HERALD_PG_DSN environment variable (format postgres://user:pass@host:port/dbname[?sslmode=disable]). Per Constitution §11.4.6 (no-guessing), the CLI does **not** fall back to localhost-with-quickstart-defaults — the operator must be explicit about which Postgres to mutate. A missing DSN yields a clear error naming the variable and format.

Spec vs as-built. Spec §9.6 documents a richer planned surface (migrate up [--steps N], migrate down --steps N, migrate force <version>, --yes confirmation). As of this revision only up (apply-all) and status are implemented; --steps, down, force, and validate are future HRDs. Document against the shipped behaviour above, not the spec's planned surface.

§7. The background task queue + Redis (wired in commons_infra)

The `storage.go` package doc lists "Redis tenant-namespacing client" and the project README lists a "River queue" as part of the storage layer. As of this revision the **live wiring of both lives one layer up, in commons_infra** (`boot.go / clients.go`), driven by `QuickstartBoot.Up()`:

- **Task queue.** `boot.go` constructs `bg.NewPostgresTaskQueue(repo, queueLogger)` — the upstream `digital.vasic.background.PostgresTaskQueue` bound to Herald's local `pgxTaskRepository` (`commons_infra/task_repository.go`) over the migration `000009/000013/000014` tables. This is the §11.4.74 extend-don't-reimplement seam: `Enqueue/Dequeue/Peek/Requeue/MoveToDeadLetter` logic is inherited; only the thin SQL-binding repository is Herald-owned. Spec §5.3 names the architectural backend "Postgres + River default"; the **implemented** queue is this Postgres-backed task queue over the same pool `commons_storage.Open` returns. The dead-letter plane (`MoveToDeadLetter` → `.queue.dead_letter CloudEvent` → `DeadLetterSubscriber` → sink) is wired to a real consumer per HRD-147 (no emit-into-the-void).
- **Redis.** `boot.go` opens a `redis.Config{ Addr, Password, DB }` from `HERALD_REDIS_ADDR` (default `127.0.0.1:24200`), `HERALD_REDIS_PASSWORD`, `HERALD_REDIS_DB`. Used for the idempotency `SETNX` hot path (24h TTL) and the constitution-bindings 60s read cache (`000007`).

`clients.go` exposes `Pool()` / `Queue()` / `Redis()` getters that return a live client only after `Up()` and `ErrNotBooted` otherwise. So: **commons_storage owns the pool + migrations + RLS; commons_infra owns the lifecycle that boots Postgres+Redis+Queue and hands the pool to the queue.**

§8. Environment variables

These are the storage-relevant variables actually read in source (`commons_infra/boot.go`, `pherald/cmd/pherald/migrate.go`, `quickstart/`), not invented:

Variable	Read by	Default	Meaning
HERALD_PG_DSN	pherald migrate up/status (migrate.go); also cherald store selection (cherald/cmd/cherald/main.go)	none (required for migrate)	Full Postgres URL <code>postgres://user:pass@host:port/dbname[?sslmode=...]</code> . No default.
HERALD_DB_PASSWORD	boot.go (QuickstartBoot), quickstart compose	<code>herald_dev</code> (boot) / required in compose	Postgres password for the boot container. The quickstart comp declares <code>POSTGRES_PASSWORD=\${HERALD_DB_PASSWORD:required...}</code> .
HERALD_DB_PORT	boot.go	24100	Host port for the quickstart Pos (maps <code>24100:5432</code> per spec reserved range).
HERALD_DB_NAME	boot.go		Postgres database name.

Variable	Read by	Default (compose default herald)	Meaning
HERALD_REDIS_ADDR	boot.go	127.0.0.1:24200	Redis host:port (maps 24200:6379).
HERALD_REDIS_PASSWORD	boot.go, quickstart compose	" " (boot) / required in compose	Redis password; compose runs redis-server -- requirepass \$ {HERALD_REDIS_PASSWORD} required...}.
HERALD_REDIS_DB	boot.go	0	Redis logical DB index.

Resolution order (per OPERATOR_CREDENTIALS.md + spec §3.3): explicit CLI flag (when one exists) → exported shell vars → .env fallback → compiled default. .env never overrides shell exports. See quickstart/.env.example for the canonical template (e.g. HERALD_PG_DSN=postgres://herald:...@127.0.0.1:24100/herald).

§9. How flavors consume commons_storage

Flavor binaries do **not** open pgx pools by hand — they go through commons_storage:

- pherald migrate — the operator/CI entry point. ParseDSN(HERALD_PG_DSN) → Open → RunMigrations / CurrentVersion. This is the canonical way to roll schema forward against any Postgres (§6).
- pherald serve / pherald listen **Runner** — uses OpenWithPool to get the raw *pgxpool.Pool for the pg adapters that drive pgx.Query/pgx.Exec directly, and wraps every multi-tenant access in WithTenantContext (§3).
- cherald — selects a Postgres-backed ConstitutionStore / Modeladder / audit backend when HERALD_PG_DSN is set (over ParseDSN + the shared pool), and an in-memory fallback otherwise (cherald/cmd/cherald/main.go).
- **All flavors via commons_infra.QuickstartBoot** — the daemon path boots Postgres+Redis+Queue once and shares the single pool across the store, the queue repository, and the Redis cache (§7).

The rule (spec V4 §10 layering): put shared storage code in commons_storage; flavors inherit upward and never re-open their own pool.

§10. Operator migrations runbook

Bring schema up against a target Postgres (assumes the quickstart stack from quickstart/docker-compose.quickstart.yml, or any reachable Postgres):

```
# 1. Set the DSN explicitly – no silent default (§6).
export HERALD_PG_DSN='postgres://
    herald:<password>@127.0.0.1:24100/herald?sslmode=disable'

# 2. Check the current version (0 on a fresh DB – the tracking
    table is absent yet).
pherald migrate status
# -> schema version: 0
```

```
# 3. Apply all pending migrations (idempotent – re-running
    applies nothing new).
pherald migrate up
#   -> applied 15 migration(s)           # on a fresh DB; fewer if
    some already applied

# 4. Confirm.
pherald migrate status
#   -> schema version: 15
```

Notes:

- **Idempotent.** `migrate up` only applies versions not already in `schema_migrations`; a second run prints `applied 0 migration(s)`.
- **Roles first.** Migration 000001 creates `herald_migrator` + `herald_app`; you must connect as a role allowed to `CREATE ROLE` (the bootstrap `POSTGRES_USER`, e.g. `herald`).
- **Down is not yet wired.** `pherald migrate down` returns an honest 501 (§6). To roll back today you apply the matching migrations/`NNNNNN_<name>.down.sql` manually under operator authorisation per §9.1, then delete the row from `schema_migrations` — there is no CLI shortcut yet.
- **Evidence.** Per §107.x, a migration run that ships a feature lands its transcript (the `migrate status/up` output + the resulting schema dump) under `docs/qa/<run-id>/`.

§11. Troubleshooting

`migrate up/status` **errors with "HERALD_PG_DSN environment variable must be set"**. The CLI has no silent default by design (§6). Export the DSN. Format must be `postgres://` or `postgresql://`; other schemes are rejected by `ParseDSN`.

Tenant A can see tenant B's rows (RLS not isolating). Two root causes (§3.3):

1. The table lacks `FORCE ROW LEVEL SECURITY` — owners bypass plain RLS. Confirm migration 000008 (and the per-table `FORCE` lines) applied: `SELECT relname, relforcerowsecurity FROM pg_class WHERE relrowsecurity;`
2. The query ran as a **superuser** (the bootstrap `POSTGRES_USER`), which bypasses RLS even with `FORCE`. You **MUST** go through `WithTenantContext`, which runs `SET LOCAL ROLE herald_app` first. Querying multi-tenant tables directly as the bootstrap user is the E14 bluff — fix the call site, do not "fix" it by loosening the policy.

`current_setting('app.tenant_id')` **errors / zero rows**. The GUC was never set. Every multi-tenant access **MUST** run inside `WithTenantContext` (or at minimum run `SetTenantContext(tenantID)`'s `SET LOCAL`). RLS fails-closed: no GUC → zero rows / blocked writes (spec §16).

SASL auth failure connecting to Postgres on :24100. Symptom: SASL authentication failed / password mismatch. Known causes for the quickstart stack:

- The container is up but `HERALD_DB_PASSWORD` in your shell/.env does not match what the `herald-postgres` container was first initialised with — Postgres only honours `POSTGRES_PASSWORD` on **first** volume init. A stale volume from a different password

(e.g. left over from the herald-e2e integration project) keeps the old password. Tear the volume down and re-up, or align the password.

- The Postgres container is not actually listening yet — 24100:5432 is the host mapping; ensure the container (and, on macOS, podman machine) is started before connecting.
- sslmode mismatch — local-dev uses sslmode=disable (ConfigForHerald default). A DSN forcing sslmode=require against the no-TLS quickstart Postgres fails the handshake before SASL.

`migrate status` **returns 0 against a DB that clearly has tables**. `CurrentVersion` keys on the `schema_migrations` tracking table specifically. If a schema was created by some other tool (not `RunMigrations`), there is no tracking row and `status` reports 0. Run `migrate up` to let Herald record versions (it will skip objects created with `IF NOT EXISTS`, but the tracking rows will be inserted).

`migrate up` **fails mid-way**. Each migration runs in its own transaction (`applyMigration`); a failing migration rolls back its own transaction and `RunMigrations` returns the versions applied **before** the failure plus the wrapped error naming the failing `v<N>` (`<name>`). Fix the SQL / DB state and re-run — already-applied versions are skipped.

§12. Testing notes

Tests live in `commons_storage/`:

File	Kind	Proves
<code>storage_test.go</code>	unit (no DB)	<code>SetTenantContext</code> SQL shape, basic helpers. The embedded bundle parses, every expected 000001..000015 has a paired up+down, and bodies are non-trivially short. (Its prose comment count is stale — see §4 note.)
<code>migrations_test.go</code>	unit (no DB)	The §107 E14 tenant-isolation round-trip — the canonical proof that <code>WithTenantContext</code> actually isolates tenants (the lesson in §3.3). Requires a running Postgres.
<code>storage_integration_test.go</code>	integration (real Postgres)	
<code>resource_stress_chaos_test.go</code>	stress + chaos (real Postgres)	Sustained/concurrent load + fault injection on the storage layer per §11.4.85.

```
# Unit (no external services):
```

```
go test -race -count=1 ./commons_storage/...
```

```
# Integration / stress (needs a real Postgres — e.g. the  
quickstart stack on :24100):
```

```
HERALD_DB_PASSWORD=... HERALD_REDIS_PASSWORD=... go test -race  
-count=1 ./commons_storage/...
```

Integration tests require operator-supplied credentials + a running Postgres (no fakes beyond unit tests, per Universal §11.4.27). See `docs/CONTINUATION.md` for the live-test handoff.

§13. References

- Source: `commons_storage/storage.go`, `commons_storage/postgres.go`, `commons_storage/migrator.go`, and `commons_storage/migrations/000001..000015_*.sql`.
- CLI: `pherald/cmd/pherald/migrate.go` (`pherald migrate up/status/down/validate`).
- Lifecycle / queue / Redis seam: `commons_infra/boot.go` + `commons_infra/clients.go` (`QuickstartBoot`, `bg.NewPostgresTaskQueue`, `Redis client`).
- Operator credentials: `docs/guides/OPERATOR_CREDENTIALS.md` — **Postgres (LIVE)** section (`env vars`, `resolution order`) + the `.env` template at `quickstart/.env.example`.
- Spec: `docs/specs/mvp/specification.V4.md` §9.2 (Postgres + Row-Level Security), §9.6 (Database migration tooling — planned surface), §5.3 (Queue backend — Postgres + River default), §16 (tenant isolation), §32 (inbound pipeline tables), §10 (layering).
- Tenant-isolation tests: `commons_storage/storage_integration_test.go` (E14) + `commons_constitution/postgres_integration_test.go` (`TestPostgresStore_RLSTenantIsolation`, the Trap-A discovery).
- Dependencies: `digital.vasic.database` (`pgx pool` + `migration runner`, ... / `submodules/database`), `github.com/jackc/pgx/v5 v5.9.2`, `github.com/redis/go-redis/v9 v9.7.3`, `github.com/google/uuid` (`commons_storage/go.mod`).

Sources verified

This guide documents internal Herald source only; no external service/library online documentation was relied on. All behavioural claims are grounded in the cited source files as of 2026-05-31 (`commons_storage/storage.go` + `postgres.go` + `migrator.go` + the 15 migration `.up.sql` files; `pherald/cmd/pherald/migrate.go`; `commons_infra/boot.go` + `clients.go`; `quickstart/docker-compose.quickstart.yml` + `.env.example`).

Verified 2026-05-31: internal doc — no external online sources. The migration count (15: 000001..000015), the role names (`herald_migrator` / `herald_app`), the `FORCE ROW LEVEL SECURITY + SET LOCAL ROLE E14` lesson, the env-var names (`HERALD_PG_DSN` / `HERALD_DB_PASSWORD` / `HERALD_REDIS_PASSWORD` / `HERALD_DB_PORT=24100` / `HERALD_REDIS_ADDR=...:24200`), and the `live-vs-501 pherald migrate` subcommand split were each read directly from the source cited above. Re-verify on a new migration landing (000016+), a `pherald migrate down/validate` implementation, or an `HERALD_*` env-var change.